

Diagnóstico — Timeout en /recaudacion/consulta-caja/padron/ (carga de adeudos)

Para: Equipo de desarrollo — opensir **De:** Alfonso Sandoval (asandoval), Infraestructura y Ciberseguridad

Fecha: 27-28 de agosto de 2026

Síntoma

Al buscar un predio (u otro padrón) con adeudos en la Ventanilla de cobro (Recaudación > Caja), la pantalla se queda en “Los cargos aún se están calculando...” con polling repetido, y eventualmente el navegador reporta:

```
POST https://apisir.municipiodeoaxaca.gob.mx/recaudacion/consulta-caja/padron/
504 (Gateway Timeout)
Network Error / ERR_FAILED
```

Caso observado: predio 1010000073985 (Ana Ramírez García), búsqueda vía Ventanilla > Público general.

No es un incidente nuevo — mismo endpoint visto desde temprano el 27 de agosto

En los primeros logs de uWSGI revisados ese día (ventana ~10:05 am), ya se había observado el mismo endpoint fallando, sin que en su momento se profundizara en la causa:

```
POST /recaudacion/consulta-caja/padron/ => generated 0 bytes in 97893 msecs
POST /recaudacion/consulta-caja/padron/ => generated 1223307 bytes in 27968 msecs
```

Es decir, este problema lleva presente desde el inicio del día de producción real, con tiempos de 28 a 98 segundos ya desde entonces.

Vista y archivo involucrados

- Archivo: `opensir/recaudacion/views/cargo.py`
- Vista: `CargoViewSet` (`ViewSet` registrado bajo `consulta-caja`), acción `cargos_por_padron` (línea 742), decorada con `@action(methods=['post'], detail=False, url_path='padron', url_name='cargos_a_pagar_por_padron')` — de ahí la URL final `consulta-caja/padron/`.

Causa raíz identificada

El sistema ya cuenta con un patrón de Celery + polling para manejar consultas pesadas (el mismo patrón usado en `cargos_por_folio`, con un comentario explícito en el código que lo referencia como ejemplo — línea ~593). Sin embargo, en `cargos_por_padron` el orden de las operaciones hace que ese mecanismo llegue demasiado tarde:

```
| cargos = Cargo.objects.select_related().filter(
```

```

        content_type__model=padron_obj.__class__.__name__.lower(),
        object_id=padron_obj.id,
        saldado=False,
        entidad_id=request.user.entidad.id,
        tipo_de_cargo__canales_de_pago=canal_de_pago,
        eliminado=False
    ).exclude(estados_globales_id=5)

    recalcular_adeudos_de_cargos(cargos)          # <- SIEMPRE se ejecuta síncrono,
                                                #     sin importar el volumen

    cargos = Cargo.objects.select_related().filter(...) # <- vuelve a consultar
                                                        #     lo mismo (redundante)

    if timeout or len(cargos) > 100:              # <- la decisión de ir a Celery
        ...delega a consulta_adeudos_task...      #     ocurre DESPUÉS del recálculo
    else:
        ...procesa síncrono...

```

recalcular_adeudos_de_cargos(cargos) se ejecuta antes de que el código decida si debe delegar a una **tarea asíncrona**. Para un padrón con muchos años de adeudo acumulado (cientos o miles de cargos/recargos/actualizaciones/gastos pendientes), ese recálculo síncrono ya consume el tiempo que el mecanismo de Celery estaba diseñado para evitar — el `if timeout or len(cargos) > 100` nunca llega a tiempo de rescatar la petición.

Segundo hallazgo, dentro de la misma función (recalcular_adeudos_de_cargos, línea 166)

La función recalcula 4 tipos de objetos relacionados al cargo (cargos, recargos, actualizaciones, gastos), y para cada uno que necesita recalcularse, itera **objeto por objeto** llamando a un método individual:

```

for crg_recalculo_adeudo in cargos_a_recalcular:
    crg_recalculo_adeudo.calcular_adeudo_base()

for rec_recalculo_adeudo in recargos_a_recalcular:
    rec_recalculo_adeudo.calcular_adeudo()

for act_recalculo_adeudo in actualizaciones_a_recalcular:
    act_recalculo_adeudo.calcular_adeudo()

for gst_recalculo_adeudo in gastos_a_recalcular:
    gst_recalculo_adeudo.calcular_adeudo()

```

No se confirmó en esta sesión si `calcular_adeudo_base()/calcular_adeudo()` disparan consultas individuales a la base de datos por objeto (patrón N+1 ya visto varias veces hoy en otros endpoints), pero es la hipótesis más probable dado el patrón repetido del día. **Se recomienda perfilar esta función específicamente** (con `time.process_time()` vs `time.time()`, como se hizo con `tramites-indexados`) antes de optimizarla a ciegas.

Relación con el bug ya reportado en registrar_recargo_en_cargos

Este flujo (`cargos_por_padron`) no llama directamente a `registrar_recargo_en_cargos` (la función con el

bug de indentación / `UnboundLocalError` ya reportado en `opensir/recaudacion/utills/recargos_cargos.py` — ver bitácora de cambios). Son hallazgos independientes, pero ambos viven en el mismo módulo de cálculo de recargos/adeudos y conviene que desarrollo los revise en la misma sesión de trabajo.

Fix sugerido

1. **Mover la decisión síncrono/asíncrono ANTES del recálculo pesado.** Contar los cargos relevantes (con una query ligera, similar a como ya se hace en `cargos_por_folio` con `total_cargos = Cargo.objects.filter(...).count()` antes de decidir) y, si supera el umbral, delegar TODO el trabajo — incluyendo `recalcular_adeudos_de_cargos`— a la tarea de Celery (`consulta_adeudos_task`), no solo la construcción de la respuesta final.
2. **Revisar si `calcular_adeudo_base()`/`calcular_adeudo()` pueden optimizarse** para trabajar en bloque (bulk update / anotaciones SQL) en vez de objeto por objeto, una vez confirmado que ahí se concentra el tiempo.
3. **Eliminar la doble consulta redundante** (líneas 831-838 y 842-849 son idénticas) — de bajo impacto comparado con los puntos anteriores, pero es una query de más sin necesidad.

Estado

● Pendiente de fix. Reportado el 27-28 de agosto. Sin mitigación de infraestructura aplicable (es un problema de orden de operaciones en el código, no de recursos de servidor).